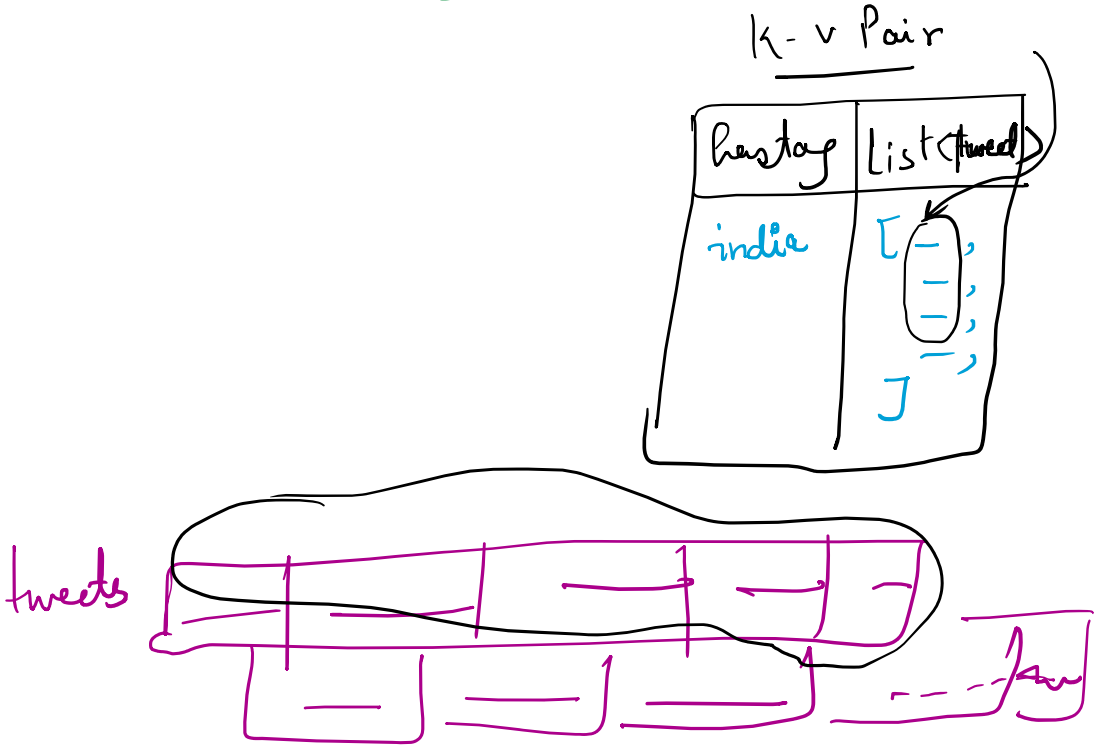
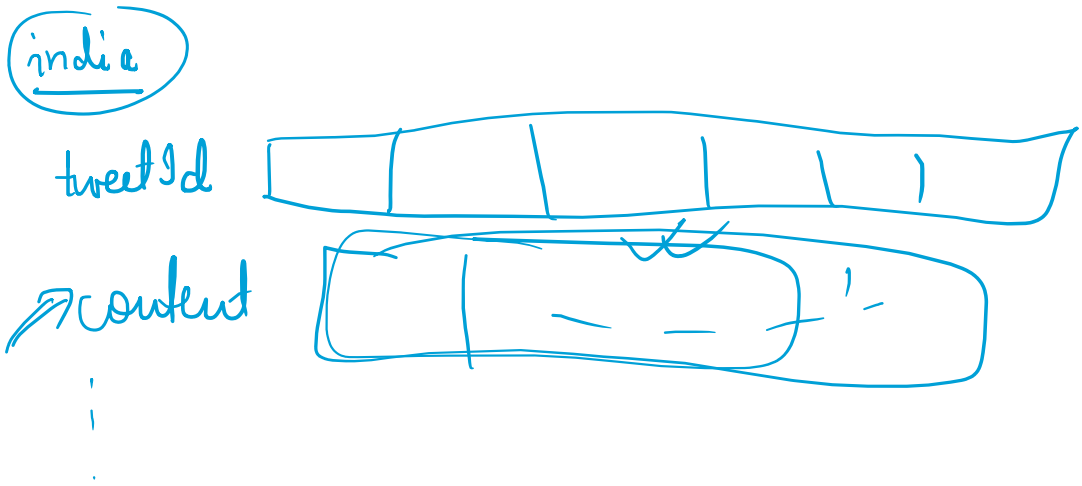
K-V	Document	Col^{n} - Family
→ When interested in value as a whole	→ When no fixed attribute	→ Query on col^{n}
→ Redis, DynamoDB	→ Mongo, Elastic search	→ Cassandra, HBase,

✱ Case

(1) Twitter Hashtag Storage

given a hashtag
fetch 10 latest tweets of that hashtag





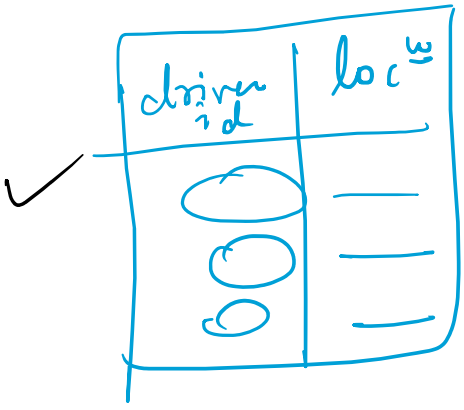
Q Get the score of a match !

Srh v/s mi ⟶ 420

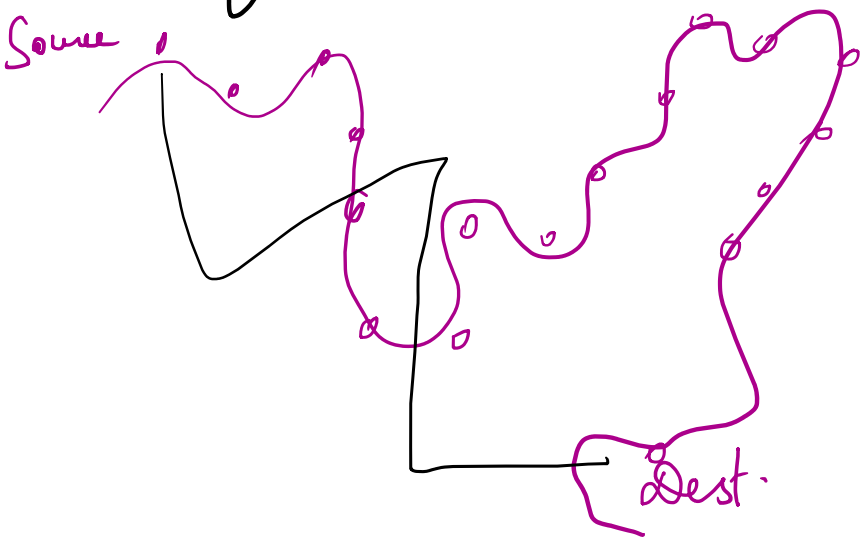




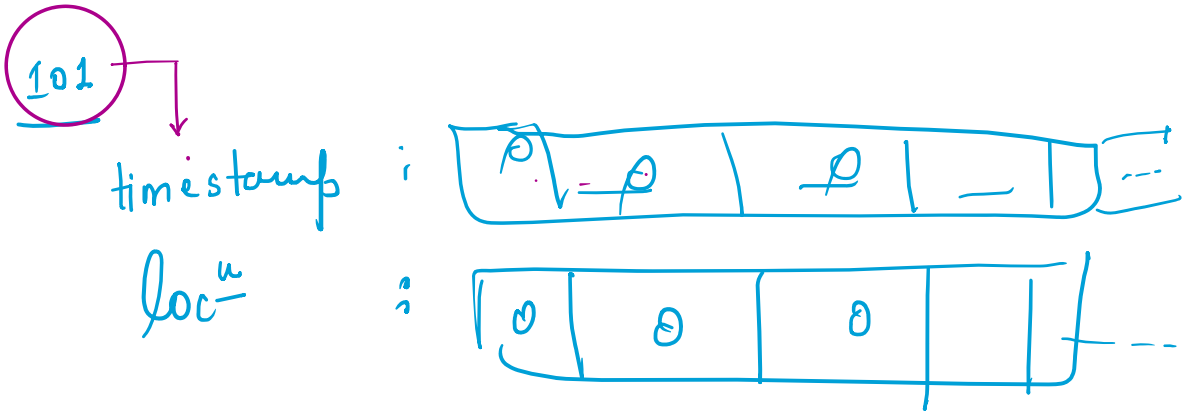
Q Current loc^{n} of a UBER driver !

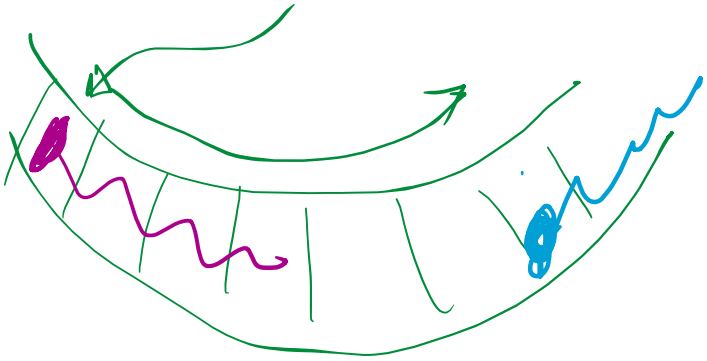


Loc^{n} history of a Uber Driver.



↳ Col^{n} family → for a driver_id





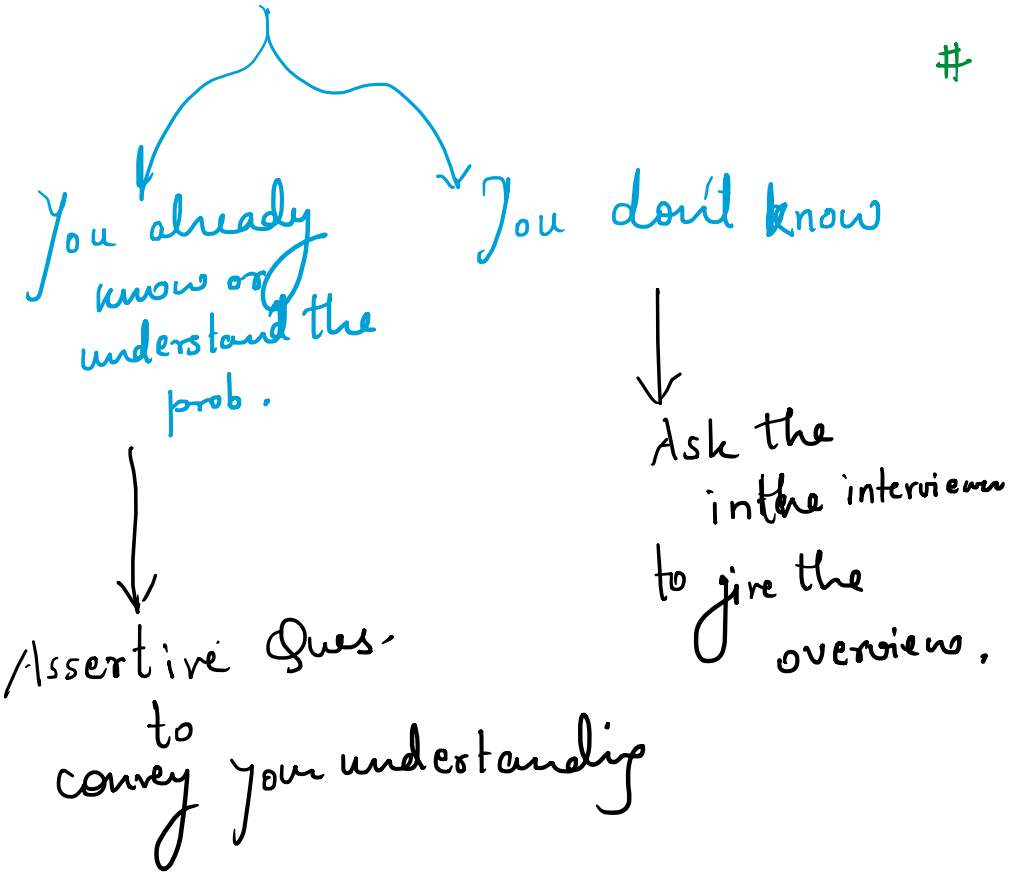
(*) Step By Step Approach.

HLD Problems

(*) Step 1 2y step Approach of HLD Problems.

0. Overview → Align w/ the interviewer.

#



1. Final-Req (Features that need to supported)

↳ don't waste too much time here.

↳ 3-5 mins.

Take a note of eventually agreed list.

↳ List down the MVP feature (Min^m viable product)

(Don't overthink / overdo) → Core Feature.

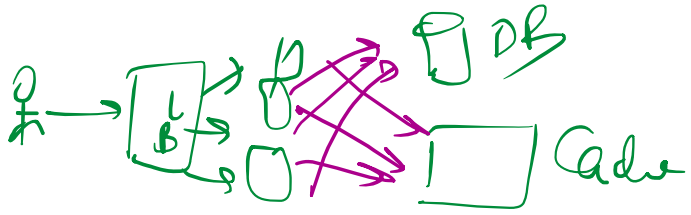
list: → (Don't overthink / overdo) → Core Feature.

2. Non-Fal Req.
   - ↳ expectations on how those features should work.
   - ↳ consistency v/s availability
   - ↳ consistency " latency.

3. Back of Envelope Estimation (BOE)
   - {Estimation of Scale}
   - ↳ Peak QPS
   - ↳ Size of data
   - ↳ Peak N/W BW
     - → Do you need sharding
     - → Read Heavy or Write Heavy

4. List APIs

5. Architecture Diag.



6. Deep Dive
   - (brainstorming on a particular component of the system)
   - How will I handle write queries
   - How will you handle recovery etc.

9:36 pm

* System Design — WhatsApp Messenger

* Overview → Messaging Appl^{n} for diff. media.
  - Text

* F^{al} Req.
  1. Send & Receive messages.
  2. Only text messages

1.)

2.) Only text messages

3.) 1:1 chatting

4.) Retention policy of 30 days.

5.) Messages are stored in backend ✗

or only on device.

6.) Get my Recent Conversations

7.) Cx can store chats on their cloud storage =



(X) Non-fᵘⁿᶜ Req.

↳ Consistency ✗ v/s Availability.

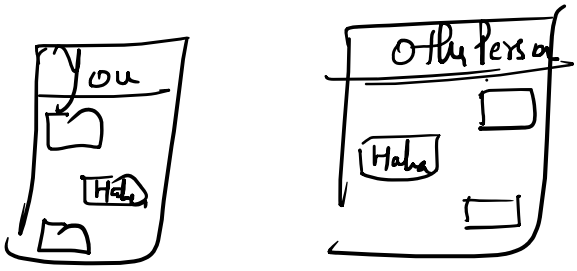
↳ Consistency ✓ v/s Latency

Consistency in context of WA Messenger

Consistency in context of WA Messenger

↳ (1.) Send Message ( ) → sender. ✓
→ Receiver ✓

↳ (2.) Order of Messages



17/10/26

⊛ Estimation of Scale (Approximation)

↓
making meaningful approximations:

- meaningful from case study perspective
- reasonable from Calculations perspective

Monthly Active Users

Assume
↳ MAU :- 1B (billion)
↳ DAU — 50% of MAU = 500M
↳ Avg. #messages/day/user — 20

⇒ (# messages/day) = $10 * 10^3$ M

$10 * 500M$ = 5000M = 5000 MB

$\Rightarrow$ # messages / day $= 10 * 10^3$ M

$= 10$ B

Avg RPS msgs. $= \frac{10\text{ B}}{24 \text{ hrs}} = \frac{10 * 10^9}{86400} \approx \frac{10 * 10^9}{10^5}$

Req. / sec.

$\approx 10^5 =$ 1 Lakh msgs. / sec.

Peak load $=$ 5 avg.

$=$ 5 Lakh msg. / sec

Storage :-

10 B / day. (B: Billion)

messages $\Rightarrow$

id $\rightarrow$ 8B (bytes)

timestamp $\rightarrow$ 8B

conv_id $\rightarrow$ 8B

user_id $\rightarrow$ 8B

$\rightarrow$ 90 B

$\approx$ 100 B

conv_id → 8B
user_id → 8B
content → ≈ 500B
receiver_id → 8B

}

per day, storage req. = 100 bytes * 10 B

= $10 * 10^2 * 10^9$ bytes

≈ 1 TB / day.

(10) =

(2)

1 kB = 2^{10} bytes = 1024 bytes ⇒ 10^3 bytes ≈ 1 kB

1 MB = 2^{10} kB = 1024 kB ⇒ 10^3 kB ≈ 1 MB

1 GB = 2^{10} MB 10^3 MB ≈ 1 GB

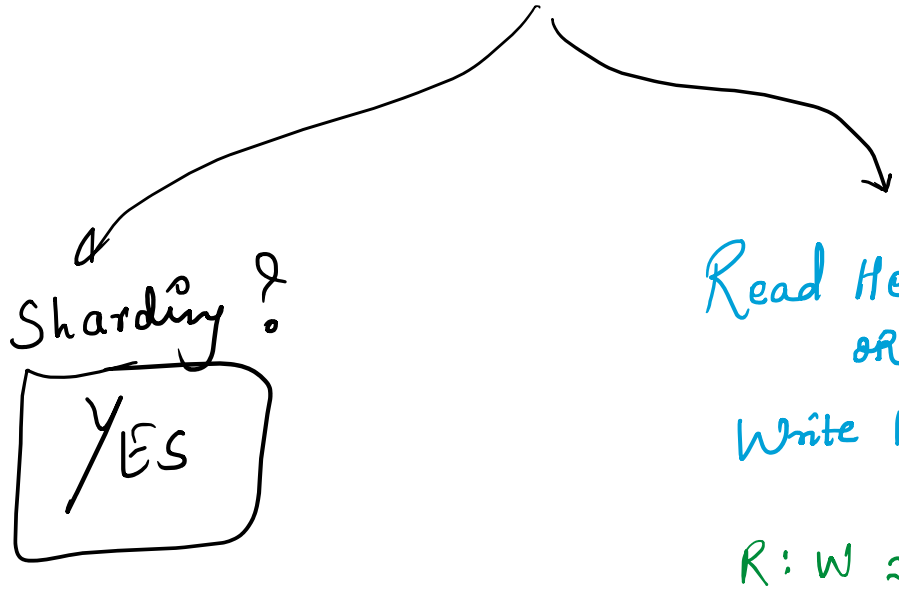
1 TB = 2^{10} GB 10^3 GB ≈ 1 TB

1 PB = 2^{10} TB 10^3 TB ≈ 1 PB

⇒ 1 yr = 365 * 1 TB ≈ 400 TB

⇒ 5 yr = 5 x 400 TB = 2 PB

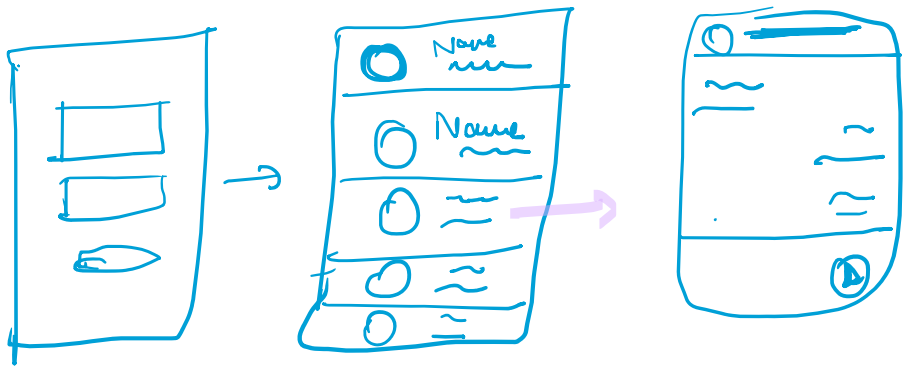
⇒ 5 yr = 5 x [illegible] = 2.5



⊛ API Design

↳ getRecentConversations (user_id)

↳ getMessages (conv_id, page_number, count)



Pagination

Limit 100 offset 1201

1201
1202
:
1300

↳ sendMessage (conv_id, user_id, content, ...)

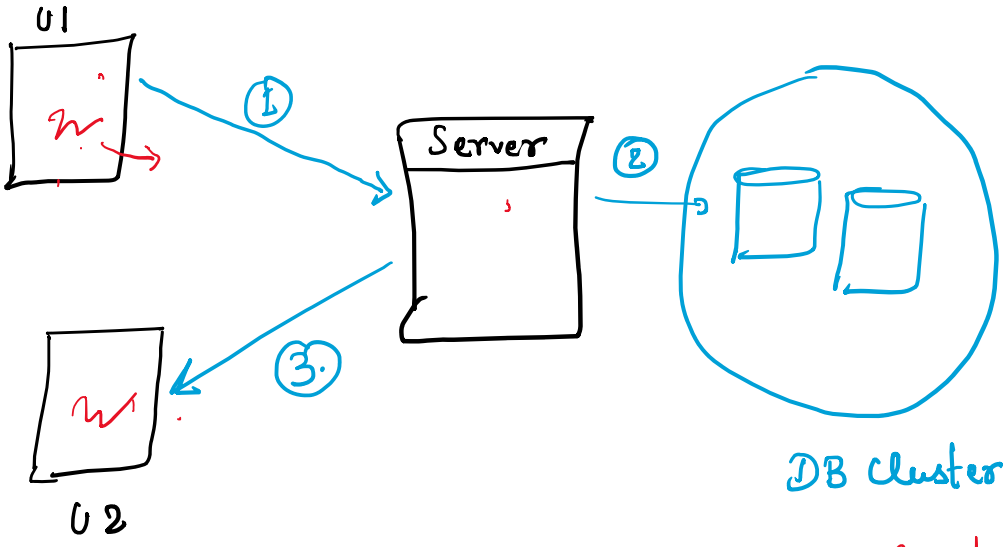
↳ creatConversation(user_id1, user_id2)

user_conversations

convId	user_id
1	101
1	108
1	110
1	120
2	130

⊛ High Level Design

p 28

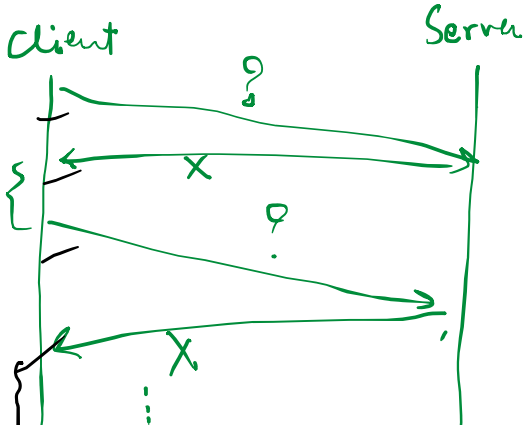


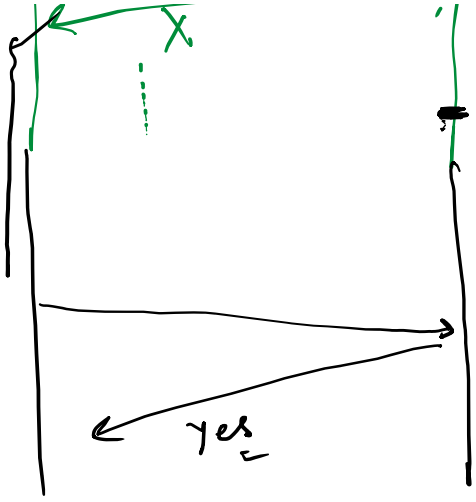
→ Server can't initiate a call to the client in client-Server Archi.

→ User always has to initiate a conversation

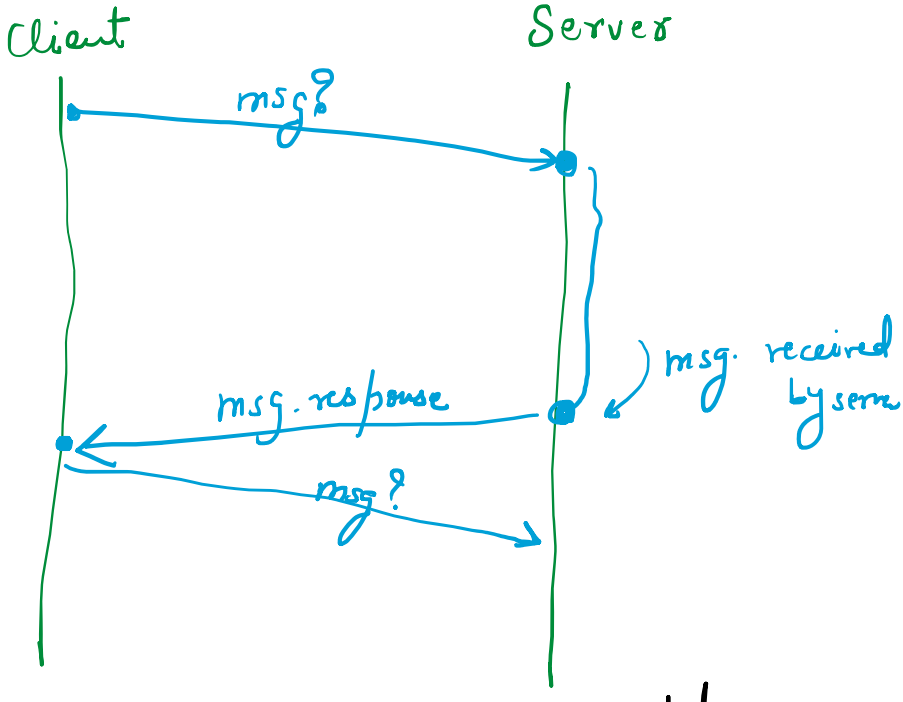
⊛ How server will send message to receiver?

1. Polling → Cons
   1.) wastage of res.
   2.) Messages aren't real time.





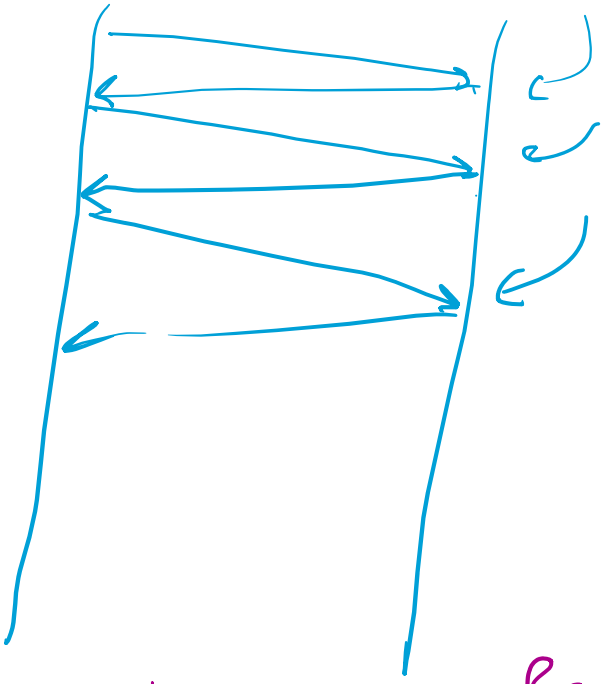
(2.) Long Polling



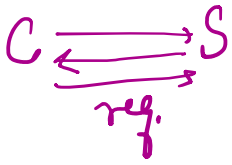
Pro → Client res. are not wasted
↳ Msgs. are real time

Con → 1.) High RAM usage @ server end
2.) Huge no. of comm^{n} req.

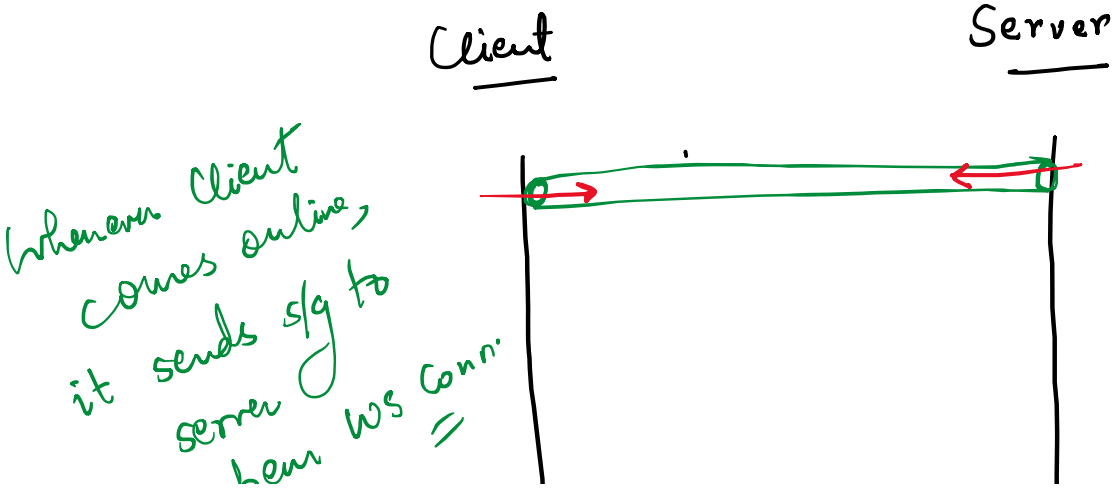
2.) Huge no. of comm^{n} req. when frequent communications happening =!



Every comm^{n} req via → 3-way Handshake s/g.



⊛ Web Socket



server ... to open WS ...

a dedicated 2-way comm^n channel is created

Pro :- Create conn. once

Con :- RAM usage is high
(on Server side)

